PROJECT REPORT

A Report on implementation of Database Management System in Real world Application

BACHELORS IN TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

BY

VAIBHAV MISHRA & UJJWAL KUMAR SRIVASTAV

Regd. Nos.: CSJMA21001390124, CSJMA21001390122

To

Dr. Deepak Kumar Verma (Head of Department CSE)

Computer Science & Engineering

University Institute of Engineering and Technology, CSJMU

(A State Government University)

KANPUR, UTTAR PRADESH, 208024

2021 – 2025

Table of Contents

1. Acknowledgement
2. Abstract
3. Introduction
4. System Overview
5. Technology Stack
6. Database Architecture
7. Implementation
   - Android Application
   - Backend Server
8. Screenshots
9. Code Snippets
10. Future Enhancements
11. Conclusion

Acknowledgment

We extend heartfelt gratitude to UIET Kanpur for the transformative experience during the Coaching Management System Project.

Mentorship:

Special thanks to Dr. Deepak Kumar Verma, Head of the Department, for invaluable mentorship, guiding me through the complexities of software engineering.

Project Experience:

Gratitude to mentors, instructors, and peers for contributing to my knowledge and providing a rich learning experience.

Collaborative Spirit:

Appreciation to fellow participants for fostering a collaborative and vibrant learning environment, elevating the project.

Support System:

Heartfelt thanks to my family and friends for unwavering support, allowing me to fully engage in the project.

Educational Foundation:

Acknowledgment to UIET Kanpur for the foundational knowledge in software engineering.

Impact:

This project expanded my technical skills and deepened my understanding of software engineering in the educational context.

Thank you

Vaibhav Mishra & Ujjwal Kumar Srivastav

B.Tech CSE

Abstract

The Coaching Management System is a dynamic software solution developed to optimize the operational and educational facets of coaching institutes. This project aims to streamline administrative processes, enhance communication, and provide a cohesive platform for students, instructors, and administrators.

Features

Student Management: Efficient record-keeping of student details, enrolment history, and payment information.

Course and Batch Management: Creation and coordination of courses, batches, and schedules.

Attendance Tracking: Monitoring and recording student attendance for performance evaluation.

Examination and Results: Management of exam schedules, grading criteria, and result records.

Payment Management: Transparent handling of student payments for financial organization.

User Authentication and Authorization: Secure access control mechanisms for data protection.

Database Architecture

The system employs a relational database structure with tables such as Students, Courses, Batches, Attendance, Exams, Results, Payments, and Users. Foreign key relationships ensure data integrity and efficient retrieval.

Android Implementation

The Android application serves as an intuitive interface, offering secure login, role-based access, and tailored dashboards for students, instructors, and administrators. It empowers users with tools for attendance tracking, result input, and comprehensive system control.

This project bridges the gap between administrative efficiency and educational excellence, providing a holistic solution for coaching institutes.

Introduction: Coaching Management System

Overview

The Coaching Management System is a comprehensive software solution designed to streamline and enhance the administrative and educational processes within a coaching institute. The system caters to the needs of students, instructors, and administrators, providing a centralized platform for efficient management and communication.

Key Features

Student Management: Allows administrators to maintain student records, including personal details, enrolments history, and payment information.

Course and Batch Management: Enables the creation and management of courses, batches, and schedules, ensuring effective coordination between instructors and students.

Attendance Tracking: Provides a mechanism for tracking student attendance, helping instructors and administrators monitor participation and performance.

Examination and Results: Manages exam schedules, grading criteria, and records results, aiding in the assessment and evaluation of student performance.

Payment Management: Facilitates the handling of student payments, ensuring a transparent and organized financial system for both students and administrators.

User Authentication and Authorization: Implements secure user authentication and authorization mechanisms to safeguard sensitive information.

Database Architecture

Tables:

Students: Stores information about individual students, including personal details, enrolments dates, and payment history.

Courses: Manages details about different courses offered, including their duration, fee structure, and assigned instructors.

Batches: Connects courses to specific batches, tracking start and end dates, schedules, and classroom information.

Attendance: Records student attendance in various batches, associating it with specific dates and status indicators.

Exams and Results: Manages exam details, including types, dates, and topics. Stores results with associated grading information.

Payments: Tracks student payments, recording payment dates, amounts, and payment methods.

Users: Stores information about system users, including administrators, instructors, and other authorized personnel.

Relationships:

Foreign Key Relationships: Establishes connections between tables, ensuring data integrity and facilitating efficient data retrieval.

Implementation in Android:

The Android application serves as the user interface for the Coaching Management System. It provides a user-friendly experience for students, instructors, and administrators through features such as:

Authentication: Secure login and user role-based access.

Student Dashboard: Access to personal details, attendance records, exam results, and payment history.

Instructor Tools: Attendance marking, result input, and batch management.

Administrator Panel: Comprehensive control over student and course management, financial tracking, and system configuration.

System Overview

The Coaching Management System is a comprehensive software solution developed to streamline and enhance the administrative and educational processes within a coaching institute. The system caters to the needs of students, instructors, and administrators, providing a centralized platform for efficient management and communication.

Key Objectives

1. **Efficient Student Management:**
   - Maintain detailed records of students, including personal information, enrolments history, and payment details.

2. **Course and Batch Management:**
   - Create and manage courses, batches, and schedules to facilitate effective coordination between instructors and students.

3. **Attendance Tracking**:
   - Implement a mechanism for tracking student attendance, enabling instructors and administrators to monitor participation.

4. **Examination and Results Management:**
   - Manage exam schedules, grading criteria, and record results to assess and evaluate student performance.

5. **Payment Management:**
   - Facilitate transparent and organized financial transactions, tracking student payments and ensuring financial accountability.

6. **User Authentication and Authorization:**
   - Implement secure user authentication and authorization mechanisms to safeguard sensitive information.

Unique Features

1. **Role-Based Access Control:**
   - Users are assigned roles (Administrator, Instructor, Student) with specific permissions, ensuring data security.

2. **Real-time Notifications:**
   - Notify students about important announcements, upcoming exams, and payment reminders in real-time.

3. **Comprehensive Reporting:**
   - Generate detailed reports on student performance, attendance, and financial transactions for informed decision-making.

4. **Secure Password Storage**
   - Utilize BCrypt hashing to securely store and validate passwords, enhancing system security.

Target Users

The system caters to the following user roles:

1. **Administrator:**
   - Comprehensive control over student and course management, financial tracking, and system configuration.

2. **Instructor:**
   - Tools for attendance marking, result input, and batch management.

3. **Student:**
   - Personalized dashboard with access to attendance records, exam results, and payment history.

Database Architecture

Database Design

Here is the brief overview of the database schema.

Tables

1. **Students**
   - Columns: student_id, first_name, last_name, date_of_birth, phone, email, address, enrollment_date, payment_history

2. **Courses**
   - Columns: course_id, course_name, description, syllabus, duration, fee_structure, instructors

3. **Batches**
   - Columns: batch_id, course_id (foreign key), start_date, end_date, schedule, instructors, classroom

4. **Attendance**
   - Columns: attendance_id, student_id (foreign key), batch_id (foreign key), date, status, remarks

5. **Exams**
   - Columns: exam_id, course_id (foreign key), exam_date, exam_type, topics, grading_criteria

6. **Exam_Results**
   - Columns: result_id, student_id (foreign key), exam_id (foreign key), marks_obtained, grade, remarks

7. **Payments**
   - Columns: payment_id, student_id (foreign key), payment_date, amount, payment_method, status

8. **Users**
   - Columns: user_id, first_name, last_name, username (unique), password_hash, role

9. **Notifications**
   - Columns: notification_id, student_id (foreign key), message, date_time, read_status

10. **Enrollments**
    - Columns: enrollment_id, student_id (foreign key), batch_id (foreign key), enrollment_date, enrollment_status

11. **Tests**
    - Columns: test_id, course_id (foreign key), question, opt_a, opt_b, opt_c, opt_d, correct_ans, user_ans

Relationships

1. **Foreign Key Relationships**
   - Ensures data integrity and efficient data retrieval.
   - Examples: student_id in Attendance references student_id in Students.

Database Implementation

MySQL Database

MySQL is used as the relational database management system.

Tables are normalized to eliminate redundancy and ensure efficient storage.

JDBC (Java Database Connectivity)

JDBC is employed for connecting the Java backend server to the MySQL database.

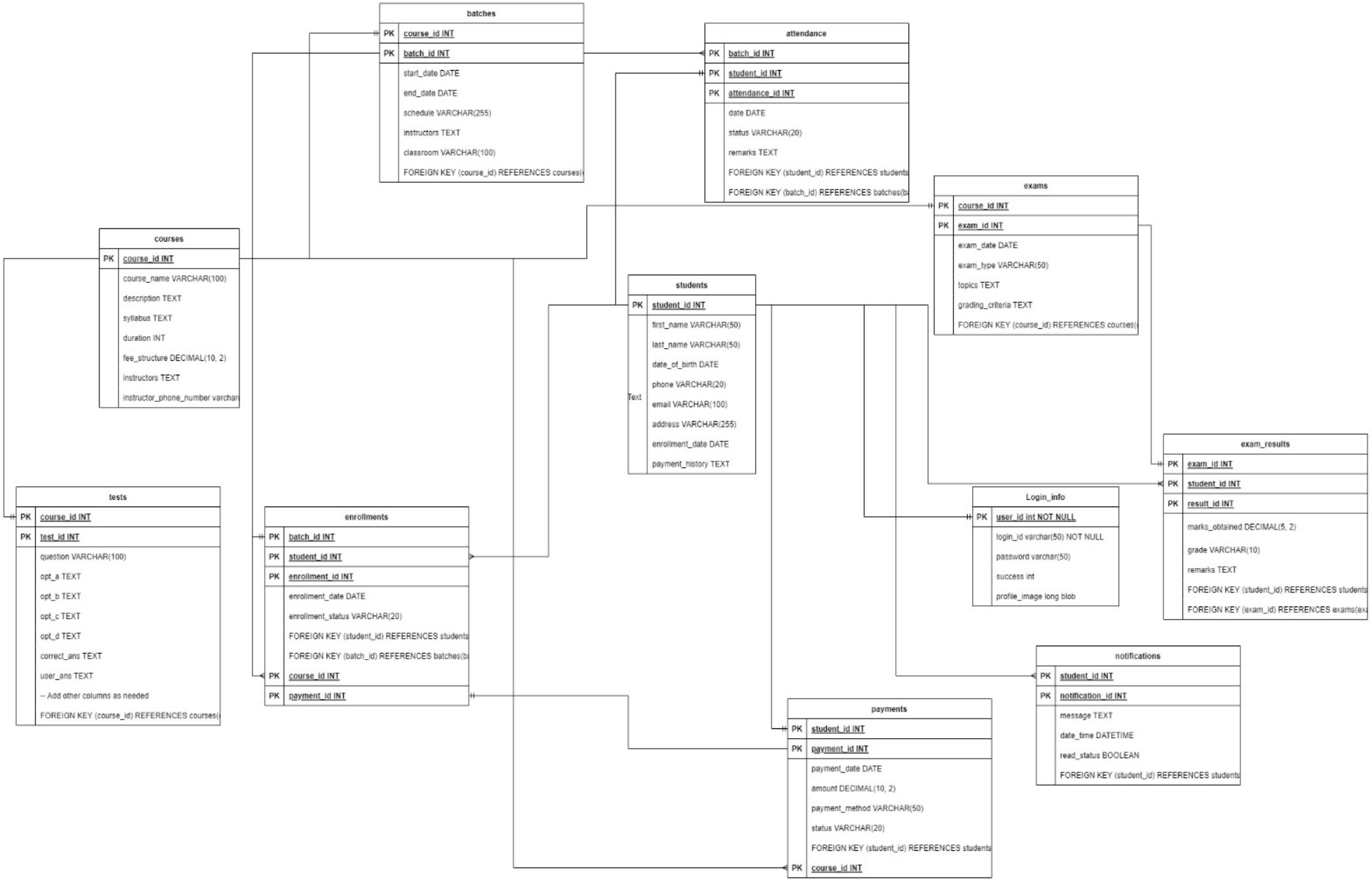
Prepared statements and transactions are used for secure and efficient data manipulation.

Security Measures

Sensitive data, such as passwords, is stored securely using hashing (BCrypt).

Access control is enforced through role-based permissions in the Users table.

E-R Diagram : Coaching Management System



Implementation

Android Application:

Structure and Architecture

The Android application follows a standard MVC architecture. The codebase is organized into packages for activities, fragments, adapters, models, and utilities.

User Interface

The user interface is designed with a clean and intuitive layout. Activities and fragments are created to handle various aspects of student, instructor, and administrator interactions.

Authentication

User authentication is implemented securely using hashed passwords stored in the database. Role-based access control ensures that each user has the appropriate permissions.

Functionality:

1. **Student Dashboard**
   - Displays personal details, attendance records, exam results, and payment history.
   - Provides an intuitive navigation flow for easy access to different sections.
2. **Instructor Tools**
   - Attendance marking for batches.
   - Input and management of exam results.
   - Batch management for instructors.
3. **Administrator Panel**
   - Comprehensive control over student and course management.
   - Financial tracking, including payment records and outstanding balances.
   - System configuration options.

Backend Server

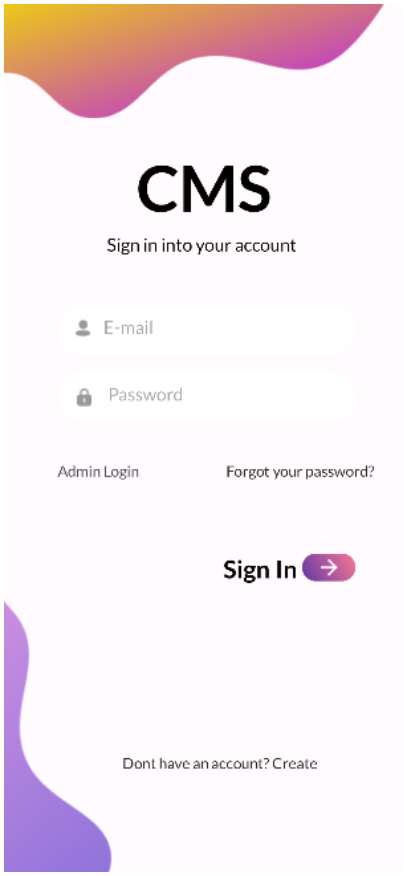
1. **Server-Side Logic:**
   - The backend server is implemented in Java and MySQL Workbench. It handles requests from the Android application, processes them, and interacts with the MySQL database.

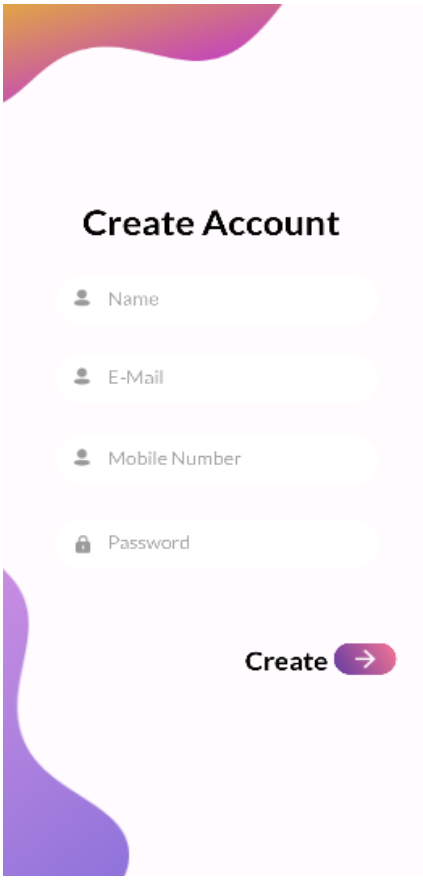
2. **Database Interaction**
   - JDBC is used to establish a connection with the MySQL database. Prepared statements and transactions are employed for secure and efficient data manipulation.

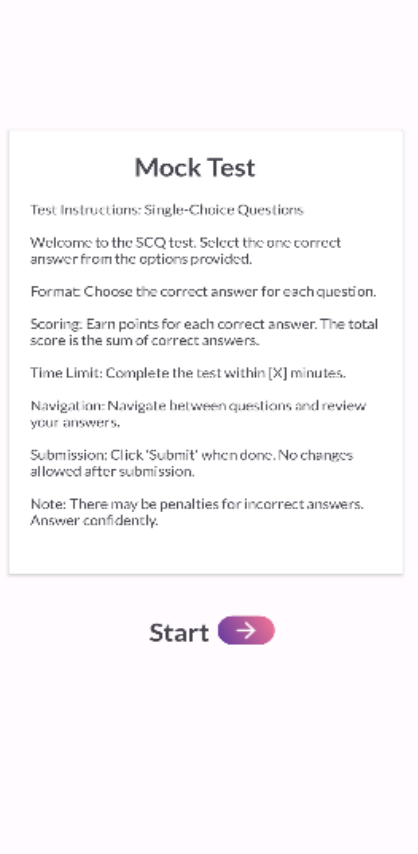
3. **Security Measures**
   - Sensitive data, such as passwords, is stored securely using hashing (BCrypt). Secure communication is ensured through HTTPS.

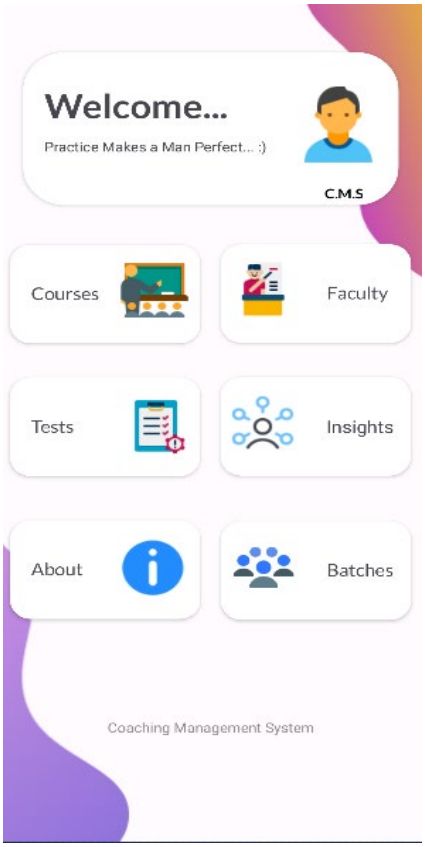
4. **Error Handling**
   - The server is equipped with robust error-handling mechanisms, providing appropriate HTTP responses for different scenarios.

Screenshots









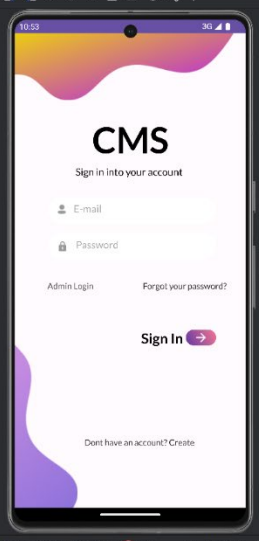
Code Snippets

```
                runOnUiThread(new Runnable() {
                    @Override
                    public void run() { profile.setImageBitmap(newBitmap); }
                });
            }
            Pair<String,Integer> data= display();
             Log.i( tag: "pair", msg: " "+data);
            runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    name.setText(data.first.toString());
                    uid.setText("Registration ID: "+data.second.toString());
                }
            });
        }

    }).start();
    profile.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            openImagePicker();

        }
    });


    //Log.i("Values","name "+ data[0].first+" "+"id "+ data[0].second);


    navView.setNavigationItemSelectedListener(new NavigationView.OnNavigationItemSelectedListener() {
        1 usage
        @Override
```

```
    1 usage
    private static Pair<String,Integer> display() {


        String user=val;
        System.out.println("User "+user);
        String res_name="xyz";
        int res_id=-1;
        //int studentID=0;
        try {

            Connection connection = DatabaseManager.getConnection();
            Statement statement = connection.createStatement();
            String query = "SELECT user_id FROM login_info WHERE login_id=?";
            PreparedStatement prep = connection.prepareStatement(query);
            prep.setString( parameterIndex: 1,user);
            ResultSet resultSet = prep.executeQuery();
            int id=-1;
            while(resultSet.next()) {
                id = resultSet.getInt( columnLabel: "user_id");
                System.out.println("ID: "+id);
            }
            String query1 = "SELECT first_name FROM students WHERE student_id=?";
            PreparedStatement prep1 = connection.prepareStatement(query1);
            prep1.setInt( parameterIndex: 1,id);
            ResultSet res = prep1.executeQuery();
            String std_name = "null";
            while(res.next()){
                std_name=res.getString( columnLabel: "first_name");
            }
            student= std_name;
```

```
pass=pass_val.getText().toString();

System.out.println(login_id+" "+pass);

final Boolean[] status = {false};
bar.setVisibility(View.VISIBLE);
Thread t1 = new Thread(new Runnable() {
    @Override
    public void run() {
        status[0] =valid_log(login_id,pass);
        if(status[0]) {
            Log.i( tag: "MY_APP", msg: "Already Logged In...");

            runOnUiThread(new Runnable() {
                @Override
                public void run() { bar.setVisibility(View.INVISIBLE); }
            });
            LocalDate date = LocalDate.now();
            String current_date= DateTimeFormatter.ofPattern("yyyy-MM-dd").format(date);
            attendance(current_date,login_id);

            Intent intent = new Intent( packageContext: MainActivity.this, Dashboard.class);
            startActivity(intent);
            val=login_id;
            //Toast.makeText(MainActivity.this, "Already Logged In...", Toast.LENGTH_LONG).show();
        }
        else{
            bar.setVisibility(View.INVISIBLE);
            ViewCompat.postOnAnimation(Objects.requireNonNull(MainActivity.this.getCurrentFocus()), new Runnable
                @Override
                public void run() {
```

```
Colors.applyToActivitiesIfAvailable(this.getAp

.setOnClickListener(new View.OnClickListener() {
erride
lic void onClick(View v) {
 login_id=email_val.getText().toString();
 pass=pass_val.getText().toString();

 System.out.println(login_id+" "+pass);

 final Boolean[] status = {false};
 bar.setVisibility(View.VISIBLE);
 Thread t1 = new Thread(new Runnable() {
     @Override
     public void run() {
         status[0] =valid_log(login_id,pass);
         if(status[0]) {
             Log.i( tag: "MY_APP", msg: "Already Logged In...");

             runOnUiThread(new Runnable() {
                 @Override
                 public void run() { bar.setVisibility(View.INVI
             });
             LocalDate date = LocalDate.now();
             String current_date= DateTimeFormatter.ofPattern("y
             attendance(current_date,login_id);

             Intent intent = new Intent( packageContext: MainActivit
             startActivity(intent);
             val=login_id;
             //Toast.makeText(MainActivity.this, "Already Logged
```



Future Enhancements

Integration of Communication Features

Implement real-time communication features, such as chat or messaging, to facilitate seamless communication between students, instructors, and administrators.

Enhanced Analytics and Reporting

Develop advanced analytics tools to provide deeper insights into student performance, attendance trends, and financial patterns.

Mobile App Features

Extend the Android application with additional features, such as push notifications, offline access, and a more intuitive user interface for a better mobile experience.

Integration with Learning Management Systems (LMS)

Explore the integration of the Coaching Management System with popular Learning Management Systems to leverage additional educational resources and functionalities.

Machine Learning for Predictive Analysis

Integrate machine learning algorithms to predict student performance trends, helping instructors and administrators identify areas for improvement.

Conclusion

The Coaching Management System represents a significant milestone in the convergence of software engineering and educational management. As the system continues to evolve, it is poised to address the dynamic needs of coaching institutes by providing a robust platform for efficient administration and enhanced learning experiences.

The development and implementation of this system have not only showcased the capabilities of Java, JDBC, and MySQL in Android application development but have also demonstrated the potential for technology to revolutionize traditional educational management practices.

In conclusion, the Coaching Management System stands as a testament to the synergy between technology and education. The envisioned future enhancements aim to further elevate the system's capabilities and contribute to the ongoing advancement of coaching institute management solutions.

We express our gratitude to all contributors and stakeholders involved in the development of this system, and we look forward to its continued growth and positive impact on the educational landscape.

Thank You

Vaibhav Mishra & Ujjwal Kumar Srivastav

B.Tech CSE